

Q.1) create a simple project using any programming language and perform some operation on the project using git. Make your project as a git repository add your file staging area and commit changes with a descriptive message.

Here is the **perfect simple answer** for your practical/viva exam.

Easy, clear, and step-by-step.

 Q.1) Create a simple project using any programming language and perform some operations using Git

Below is a **simple project using Python** and then **Git commands** to perform all required operations.

 Step 1: Create a Simple Project

Create a folder named **myproject** and inside it create a Python file:

File: main.py

```
print("Hello, Git! This is my first project.")
```

 Step 2: Open Terminal (CMD / Git Bash)

Go to your project folder:

```
cd myproject
```

 Step 3: Initialize Git Repository

```
git init
```

This will create a hidden `.git` folder and your project becomes a **Git repository**.

 Step 4: Check Status

```
git status
```

You will see `main.py` as an untracked file.

 Step 5: Add File to Staging Area

```
git add main.py
```

Step 6: Commit the Changes

```
git commit -m "Initial commit: Added main.py with Hello Git message"
```

A descriptive commit message is given.

Step 7: Make Some Changes

Open **main.py** and modify it:

```
print("Hello, Git! This is version 2.")
```

Step 8: Check Git Status Again

```
git status
```

Step 9: Add & Commit Again

```
git add main.py
```

```
git commit -m "Updated message to version 2"
```

Your Required Output Summary

- ✓ Created a simple project
 - ✓ Initialized a Git repository
 - ✓ Added file to staging area
 - ✓ Made commits with descriptive messages
-

Viva Notes (Short Answers)

1. What is Git?

Git is a **version control system** that keeps track of changes in files and allows multiple people to work on a project.

2. What is a repository?

A repository (repo) is a **storage location** where Git stores all history of your project.

3. What is staging area?

The staging area holds the files **before** committing them.

4. What is a commit?

A commit is a **saved snapshot** of your project changes.

Summary Sheet

Git Basic Commands

Command	Purpose
git init	Creates a new Git repository
git status	Shows file status
git add filename	Moves files to staging area
git commit -m "message"	Saves changes permanently
git log	Shows commit history

2) Q.2) create a simple Java project using Maven. adding dependencies, and Configuring the project's POM file and compile code using maven tool.

Here is the **perfect, simple, exam-ready answer** with **commands + steps + small example** for Maven project creation, adding dependencies, configuring POM, and compiling code.

Q.2) Create a simple Java project using Maven, add dependencies, configure POM, and compile code

Step 1: Install Maven

Make sure Maven is installed:

```
mvn -v
```

If version is shown → Maven is installed.

Step 2: Create a Simple Maven Project

Use the Maven archetype command:

```
mvn archetype:generate -DgroupId=com.example \
-DartifactId=myproject \
-DarchetypeArtifactId=maven-archetype-quickstart \
-DinteractiveMode=false
```

This creates a folder structure like:

myproject/

├── pom.xml

└── src

 └── main

 └── java

 └── com/example/App.java

Step 3: Open the Project

Go inside the project:

```
cd myproject
```

Step 4: Edit App.java (Simple Java Code)

Location:

src/main/java/com/example/App.java

```
package com.example;
```

```
public class App {
    public static void main(String[] args) {
        System.out.println("Hello Maven Project!");
    }
}
```

Step 5: Configure POM (Add a Dependency)

Open **pom.xml** and add a dependency (example: Gson library).

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>myproject</artifactId>
  <version>1.0-SNAPSHOT</version>

  <dependencies>
    <!-- Example dependency: Gson -->
    <dependency>
      <groupId>com.google.code.gson</groupId>
      <artifactId>gson</artifactId>
      <version>2.11.0</version>
    </dependency>
  </dependencies>

</project>
```

✓ This is called **configuring the POM file**.

Step 6: Download Dependencies

Run:

```
mvn dependency:resolve
```

Step 7: Compile the Code Using Maven

Use:

```
mvn compile
```

This compiles all Java files.

Step 8: Run the Project (Optional)

Maven doesn't run automatically. You can run using:

```
mvn exec:java -Dexec.mainClass="com.example.App"
```

(If exec-maven-plugin is not added, I can give you that also.)

Viva-Ready Explanation

1. What is Maven?

Maven is a **build automation tool** used for:

- building Java projects
 - managing dependencies
 - compiling programs
 - packaging into JAR/WAR
-

2. What is POM.xml?

POM means **Project Object Model**.

It contains:

- project name
 - version
 - dependencies
 - build plugins
-

3. What is a dependency?

A dependency is an **external library** your project needs.

Example: Gson for JSON handling.

4. How to compile the Maven project?

```
mvn compile
```

✨ Summary Sheet

Task	Maven Command
Create project	mvn archetype:generate
Compile project	mvn compile
Resolve dependencies	mvn dependency:resolve
Package project	mvn package
Run project	mvn exec:java

3)

Q.3) Docker and Containerization

Task 1: Dockerfile Creation and Build

- Create a Dockerfile to containerize a simple HTML web page.
- The Dockerfile should use an nginx base image and copy the HTML page to the default directory served by nginx.
- Build the Docker image and tag it appropriately.

Task 2: Running and Managing Containers

- Run the Docker container, mapping port 8080 on the host to the container's port 80.
- Confirm the web page is accessible through localhost:8080.
- Stop and remove the container after testing.

Below is the **perfect, exam-ready, simple explanation** for **Docker & Containerization – Q3**, including **commands + steps + viva notes**.

✅ Q.3) Docker and Containerization

📁 Task 1: Dockerfile Creation and Build

Step 1: Create a Simple HTML Page

Create a folder named **docker-web**

Inside it, create a file: **index.html**

index.html

```
<!DOCTYPE html>

<html>

<head>

  <title>My Docker Web Page</title>

</head>

<body>

  <h1>Hello from Docker & NGINX!</h1>

</body>

</html>
```

Step 2: Create a Dockerfile

Inside the same folder create a file named **Dockerfile** (no extension).

Dockerfile

```
# Use nginx base image
```

```
FROM nginx:latest
```

```
# Copy HTML page to nginx default directory
```

```
COPY index.html /usr/share/nginx/html/index.html
```

✓ This Dockerfile uses **nginx**

✓ Copies your HTML file to the default nginx web directory.

Step 3: Build the Docker Image

Open terminal inside the project folder and run:

```
docker build -t my-nginx-site .
```

✓ my-nginx-site is the name (tag) of your Docker image.

Task 2: Running and Managing Containers

Step 1: Run the Docker Container

Map port **8080** → **80** (nginx default port):

```
docker run -d -p 8080:80 --name mywebcontainer my-nginx-site
```

- -d → run in background
 - -p 8080:80 → host:container port mapping
 - --name → container name
-

Step 2: Test the Web Page

Open browser and visit:

<http://localhost:8080>

You will see:

"Hello from Docker & NGINX!"

Step 3: Stop the Container

```
docker stop mywebcontainer
```

Step 4: Remove the Container

```
docker rm mywebcontainer
```

Viva Ready Notes

1. What is Docker?

Docker is a platform that creates lightweight, portable containers that run applications.

2. What is a Dockerfile?

A Dockerfile is a script with instructions to build a Docker image.

3. What is an image?

An image is a blueprint used to create containers.

4. What is a container?

A running instance of an image.

5. Why use NGINX?

NGINX is a lightweight, fast web server ideal for hosting static HTML pages.

Summary Sheet

Task	Command
Build Image	<code>docker build -t my-nginx-site .</code>
Run Container	<code>docker run -d -p 8080:80 --name mywebcontainer my-nginx-site</code>
Stop Container	<code>docker stop mywebcontainer</code>
Remove Container	<code>docker rm mywebcontainer</code>

Q.4) create a simple project And push on remote server (like github) using git. and perform some operation. And displays a chronological history of commits.

1) Create the project files (use an editor / file explorer)

1. Open your code editor (VS Code / Notepad++) or File Explorer.
2. Create a folder named simple-project.
3. Inside it create files using the editor:
 - `app.py` — add:
`print("Hello from Simple Project")`
 - `README.md` — add a short project description.
 - `.gitignore` — add `__pycache__` / (and other ignores you need).

Important: Do these file creations and edits inside your editor — not via terminal.

2) Initialize Git repository (terminal commands only)

Open the terminal **inside the project folder** (or open terminal and `cd` into it), then run:

`git init`

3) Stage files and make the first commit

```
git add .
```

```
git commit -m "Initial commit: add app.py, README and .gitignore"
```

(You edited/created files in your editor — git add simply stages them.)

4) Make a second change (edit in editor) and commit

1. Open app.py in your editor and add/change code (for example, add a comment or extra print).
2. Save the file in the editor.
3. Then in terminal:

```
git add app.py
```

```
git commit -m "Add comment and minor update to app.py"
```

5) Create a remote repo on GitHub (use website or GitHub CLI)

- **Website method (recommended for exams):** Go to GitHub → New repository → name it simple-project → Create.
 - Copy the repository URL shown by GitHub (HTTPS or SSH).
-

6) Add remote and push to GitHub

Replace URL below with your repo URL:

```
git branch -M main
```

```
git remote add origin URL
```

```
git push -u origin main
```

7) Do a branching workflow (edit in editor, commit via terminal)

1. Create a branch:

```
git checkout -b feature/add-version
```

2. Edit app.py in your editor (e.g., append print("Version 1.0")) and save.
3. Stage & commit:

```
git add app.py
```

```
git commit -m "Add version print to app.py"
```

```
git push -u origin feature/add-version
```

4. Merge back to main:

```
git checkout main
```

```
git merge feature/add-version -m "Merge feature/add-version"
```

```
git push origin main
```

5. (Optional) Delete branch remotely:

```
git push origin --delete feature/add-version
```

8) Show chronological history of commits (oldest → newest)

To print commits **oldest first** (true chronological order):

```
git log --reverse --pretty=format:'%h %ad | %s [%an]' --date=short
```

- %h = short hash
- %ad = author date (YYYY-MM-DD)
- %s = commit message
- %an = author name

Example output:

```
b1f2a3d 2025-11-25 | Initial commit: add app.py, README and .gitignore [Your Name]
```

```
c4d5e6f 2025-11-25 | Add comment and minor update to app.py [Your Name]
```

```
a7b8c9d 2025-11-25 | Add version print to app.py [Your Name]
```

If you prefer a compact graph (newest → oldest):

```
git log --oneline --decorate --graph --all
```

9) Verify on GitHub

- Open your repository page on GitHub → **Commits** to view commit list (GitHub shows newest → oldest).
 - For chronological (oldest→newest) view locally use the --reverse command above.
-

10) Quick viva notes (short)

- **How did you create files?** Using an editor (VS Code / Notepad++) — not terminal.
 - **How to stage?** git add <file> or git add .
 - **How to commit?** git commit -m "descriptive message"
 - **How to push?** git push -u origin main
 - **Chronological commits?** git log --reverse ... prints oldest → newest.
-

Sure — nicely formatted Q&A answers for Q.5–Q.10. I kept each answer concise, exam-ready, and step-by-step. I avoided terminal file-creation tricks (you create files in your editor) and used only standard commands for Git / Maven / Docker / Jenkins where needed.

Q.5) Applying CI/CD Principles to Web Development Using Jenkins, Git, and a Local HTTP Server (Apache/Nginx)

A:

Goal: Automate build → test → deploy for a simple web project using Git + Jenkins + local Nginx/Apache.

Steps (high level):

1. Prepare project (local):

- Create a web project folder in your editor: index.html, optional CSS/JS.
- Initialize Git:
- git init
- git add .
- git commit -m "Initial web project"
- Push to remote (GitHub/GitLab/Bitbucket) following the remote provider UI or CLI.

2. Install Jenkins (local or server):

- Install Jenkins and open its web UI (default http://localhost:8080).
- Install recommended plugins (Git, Pipeline, SSH, etc.).

3. Create a Jenkins pipeline job:

- In Jenkins, create a new *Pipeline* job and connect it to your Git repository (provide repo URL and credentials).

- Use a Jenkinsfile in repo (preferred) or inline pipeline script.

4. **Sample Jenkinsfile (declarative)**

```
5. pipeline {
6.   agent any
7.   stages {
8.     stage('Checkout') {
9.       steps {
10.        checkout scm
11.      }
12.    }
13.    stage('Build') {
14.      steps {
15.        // if you have build steps (npm/maven) run them here
16.        echo 'Build step (if any)'
17.      }
18.    }
19.    stage('Test') {
20.      steps {
21.        // run tests if present
22.        echo 'Test step (if any)'
23.      }
24.    }
25.    stage('Deploy') {
26.      steps {
27.        // copy files to local web server directory
28.        // assumes Jenkins user has permission or uses sudo/ssh
29.        sh 'cp -r * /var/www/html/' // for Apache on many systems
30.        // or: sh 'cp -r * /usr/share/nginx/html/' for nginx
31.      }
```

```
32. }  
33. }  
34. post {  
35.     always { archiveArtifacts artifacts: '**/*', allowEmptyArchive: true }  
36. }  
37. }
```

(Place this Jenkinsfile in your repo root. Edit the cp path to match your web server's document root.)

38. Configure local HTTP server (Nginx/Apache):

- Install and run Nginx/Apache.
- Document root (common defaults):
 - Apache: /var/www/html/
 - Nginx: /usr/share/nginx/html/
- Ensure Jenkins can write to that folder (or use a deployment user/SSH).

39. Trigger & verify:

- Commit & push changes to Git → Jenkins detects change and runs pipeline (either via webhooks or polling).
- After Deploy stage, open <http://localhost> (or <http://localhost:8080> for non-default) to confirm the page served by Apache/Nginx is updated.

Viva tips (short):

- CI = continuously integrate (build + test on each commit).
- CD = continuously deliver/deploy (automatically push artifact to server).
- Jenkinsfile keeps pipeline as code and versioned with the project.

Q.6) Create a simple project, push it to GitHub, create a branch, merge it into main, and display chronological commits

A:

Steps (editor + terminal):

1. **Create project in editor** (do not use terminal for file creation). Example files: index.html, README.md, .gitignore.
2. **Initialize Git and initial commit:**

3. `cd your-project-folder`
4. `git init`
5. `git add .`
6. `git commit -m "Initial commit: add project files"`
7. **Create remote repo on GitHub** (via website) and add remote:
8. `git branch -M main`
9. `git remote add origin https://github.com/your-username/your-repo.git`
10. `git push -u origin main`
11. **Create a new branch and work there:**
12. `git checkout -b feature/add-footer`
13. # Edit files in your editor (add footer to index.html), save
14. `git add index.html`
15. `git commit -m "Add footer to index.html"`
16. `git push -u origin feature/add-footer`
17. **Merge branch into main (locally) and push:**
18. `git checkout main`
19. `git merge feature/add-footer -m "Merge feature/add-footer into main"`
20. `git push origin main`
21. **Display chronological commit history (oldest → newest):**
22. `git log --reverse --pretty=format:'%h %ad | %s [%an]' --date=short`

Notes: You can also merge via GitHub Pull Request UI — merge there and git pull to update local main.

Q.7) Configure Maven to compile code, run tests, and generate artifacts (JAR)

A:

Goal: Set up pom.xml so mvn test runs tests and mvn package produces a JAR.

Steps:

1. **Create a Maven project** (use your IDE or mvn archetype:generate if allowed outside terminal file creation).
2. **Minimal pom.xml (core parts):**


```
3. <project xmlns="http://maven.apache.org/POM/4.0.0"
4.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5.     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
6.         http://maven.apache.org/xsd/maven-4.0.0.xsd">
7.   <modelVersion>4.0.0</modelVersion>
8.   <groupId>com.example</groupId>
9.   <artifactId>myapp</artifactId>
10.  <version>1.0-SNAPSHOT</version>
11.
12.  <dependencies>
13.    <!-- JUnit for tests -->
14.    <dependency>
15.      <groupId>org.junit.jupiter</groupId>
16.      <artifactId>junit-jupiter</artifactId>
17.      <version>5.9.2</version>
18.      <scope>test</scope>
19.    </dependency>
20.  </dependencies>
21.
22.  <build>
23.    <plugins>
24.      <!-- Surefire runs unit tests -->
25.      <plugin>
26.        <groupId>org.apache.maven.plugins</groupId>
27.        <artifactId>maven-surefire-plugin</artifactId>
28.        <version>3.0.0-M7</version>
29.      </plugin>
30.    </plugins>
31.  </build>
```

32. </project>

33. Standard Maven commands:

- Compile: mvn compile
- Run tests: mvn test
- Create JAR artifact: mvn package → produces target/myapp-1.0-SNAPSHOT.jar
- Install to local repo: mvn install

34. Notes about tests:

- Put tests under src/test/java with names like AppTest.java.
- Surefire runs JUnit tests automatically during mvn test and mvn package.

Viva tips (short):

- mvn package compiles + runs tests + packages artifact.
 - Use maven-compiler-plugin to set Java version if required.
-

Q.8) Create a simple project and use Git to check status, view log history, see diffs, commit, and display chronological commits

A:

Steps (editor + terminal):

1. **Create files in editor** (e.g., app.py, README.md).
2. **Initialize and first commit:**
3. git init
4. git add .
5. git commit -m "Initial commit: add project files"
6. **Check status:**
7. git status
 - Shows untracked/modified files and branch info.
8. **Make change in editor (modify app.py), then view differences:**
9. git diff # shows unstaged changes (working directory vs last commit)
10. git add app.py
11. git diff --staged # shows staged changes vs last commit

12. Commit the change:

13. `git commit -m "Update app.py: add greeting message"`

14. View chronological history (oldest → newest):

15. `git log --reverse --pretty=format:'%h %ad | %s [%an]' --date=short`

16. Alternative compact log (graph):

17. `git log --oneline --decorate --graph --all`

Viva tips: `git diff` is your friend to review unstaged/staged changes before committing.

Q.9) Create a simple project, push to Bitbucket, create a branch, merge it into main, and show chronological history

A:

Steps (editor + terminal):

1. **Create project in editor** and commit locally:
2. `git init`
3. `git add .`
4. `git commit -m "Initial commit"`
5. **Create repository on Bitbucket** (via web UI) and copy the HTTPS/SSH repo URL.
6. **Add remote and push:**
7. `git branch -M main`
8. `git remote add origin https://bitbucket.org/your-username/your-repo.git`
9. `git push -u origin main`
10. **Create a branch, edit in editor, commit & push:**
11. `git checkout -b feature/update-readme`
12. # edit README.md in editor, save
13. `git add README.md`
14. `git commit -m "Update README with project details"`
15. `git push -u origin feature/update-readme`
16. **Merge into main (locally) and push:**
17. `git checkout main`

18. `git merge feature/update-readme -m "Merge feature/update-readme"`

19. `git push origin main`

(Or open a Pull Request in Bitbucket UI, review, and merge.)

20. **Show chronological commits (oldest → newest):**

21. `git log --reverse --pretty=format:'%h %ad | %s [%an]' --date=short`

Note: Bitbucket UI also shows commits in newest→oldest order; the `--reverse` form gives oldest→newest locally.

Q.10) Create a simple project, push to GitHub, create a branch, merge into main, display chronological commits, and pull changes locally

A:

Steps (editor + terminal):

1. **Create files in editor and commit locally:**

2. `git init`

3. `git add .`

4. `git commit -m "Initial commit"`

5. **Create GitHub repo (website) and add remote:**

6. `git branch -M main`

7. `git remote add origin https://github.com/your-username/your-repo.git`

8. `git push -u origin main`

9. **Create branch, edit, commit, and push:**

10. `git checkout -b feature/add-contact`

11. # edit files in editor and save

12. `git add .`

13. `git commit -m "Add contact section"`

14. `git push -u origin feature/add-contact`

15. **Merge branch into main:**

- Option A: Locally

- `git checkout main`

- `git merge feature/add-contact -m "Merge feature/add-contact"`

- git push origin main
- Option B: Via GitHub Pull Request:
 - Open PR on GitHub, merge there.
 - Then update local main.

16. Display chronological history (oldest → newest):

17. `git log --reverse --pretty=format:'%h %ad | %s [%an]' --date=short`

18. Pull changes to local machine (if merge done on remote or another machine):

19. `git checkout main`

20. `git pull origin main`

- This fetches and merges remote changes into your local main.

Viva tip: After a remote merge (e.g., PR merged on GitHub), always git pull locally to sync your branch.

Nice — here are **clean, exam-ready Q&A** answers for **Q.11 → Q.15**. Each question has a short goal, step-by-step actions (using your editor for file edits — **no terminal file-creation tricks**) and the exact Git/GitLab/Bitbucket/Jenkins commands you'll run in the terminal. Ready to copy-paste in your viva.

Q.11) Create a simple project, push it to a remote repository on GitLab, create a new branch, merge it into main, and display a chronological history of commits.

A:

Goal: Push project to GitLab, create branch, merge to main, show commits oldest→newest.

Steps:

1. **Create files in your editor** inside a folder gitlab-project (e.g., index.html, README.md, .gitignore).
2. **Initialize & commit locally:**
3. `cd gitlab-project`
4. `git init`
5. `git add .`
6. `git commit -m "Initial commit: add project files"`

7. **Create repository on GitLab (web UI).** Copy the repository HTTPS/SSH URL shown by GitLab.
 8. **Add remote and push main:**
 9. `git branch -M main`
 10. `git remote add origin <GITLAB_REPO_URL>`
 11. `git push -u origin main`
 12. **Create a feature branch, edit files in editor, commit & push:**
 13. `git checkout -b feature/add-footer`
 14. # edit index.html in editor -> add footer -> save
 15. `git add index.html`
 16. `git commit -m "Add footer to index.html"`
 17. `git push -u origin feature/add-footer`
 18. **Merge branch into main**
 - Option A (recommended for GitLab): open a **Merge Request** on GitLab UI from feature/add-footer → main, review and **Merge** there.
 - Option B (locally):
 - `git checkout main`
 - `git merge feature/add-footer -m "Merge feature/add-footer into main"`
 - `git push origin main`
 19. **Display chronological commit history (oldest → newest):**
 20. `git log --reverse --pretty=format:'%h %ad | %s [%an]' --date=short`
-

Q.12) Create a simple project, push it to GitLab, create a new branch, merge it into main, display chronological commits, and pull changes on your local machine.

A:

Goal: Same as Q.11, plus ensure local machine receives remote changes (pull).

Steps:

Follow steps 1–6 from Q.11. Then:

7. **If the merge was done on GitLab (Merge Request) or another machine, update your local main:**
8. `git checkout main`

9. `git pull origin main`

This pulls remote main and merges it into your local main so your local copy is up-to-date.

10. **Confirm chronological history locally:**

11. `git log --reverse --pretty=format:'%h %ad | %s [%an]' --date=short`

Q.13) (Repeat) Create a simple project, push to GitLab, create branch, merge into main, display chronological commits, and pull the changes locally.

A:

Goal: Same workflow — this answer is the identical, step-by-step process you should use.

Steps (summary):

1. Create project files in your editor.
2. `git init` → `git add .` → `git commit -m "Initial commit"`.
3. Create GitLab repo → `git remote add origin <URL>` → `git push -u origin main`.
4. `git checkout -b feature/...` → edit in editor → `git add` → `git commit` → `git push -u origin feature/...`
5. Merge via GitLab Merge Request or locally (`git checkout main`, `git merge feature/...`, `git push origin main`).
6. Pull merged changes to your local machine: `git checkout main` → `git pull origin main`.
7. Show chronological commits:
8. `git log --reverse --pretty=format:'%h %ad | %s [%an]' --date=short`

Tip: Use Merge Requests on GitLab for a professional workflow — they give review, CI checks, and an easy UI merge.

Q.14) Create CI using Webhook and deploy a project using Jenkins Execute shell.

A:

Goal: When code is pushed to Git (GitLab/GitHub/Bitbucket), a webhook notifies Jenkins to run a job which builds and deploys the project (execute shell for deployment).

High-level Steps:

1. **Prepare project & repo:**
 - Create project files in editor, push to remote (GitLab/GitHub/Bitbucket).
2. **Install & prepare Jenkins:**

- Install Jenkins and required plugins:
 - Git plugin (checkout),
 - Generic Webhook Trigger plugin **or** GitLab/GitHub plugin (for provider-specific webhooks),
 - Credentials plugin (store SSH keys or user/pass securely).
- Ensure Jenkins is reachable from your Git host (for local setups you may need ngrok or host-accessible Jenkins).

3. Create a Jenkins job (Freestyle or Pipeline):

- **Freestyle job** example:
 - **Source Code Management:** choose Git and enter repo URL + credentials.
 - **Build Triggers:** enable **Trigger builds remotely** (token) **OR** configure **Generic Webhook Trigger** to accept push events.
 - **Build → Execute shell:** add the shell steps that perform build/deploy.

4. Example contents for Jenkins Execute shell (deployment commands):

5. `#!/bin/bash`

6. `set -e`

7.

8. `# 1. Pull latest code (if Jenkins checks out, this may be optional)`

9. `# git -C /var/lib/jenkins/workspace/YourJobName pull origin main`

10.

11. `# 2. Build steps (if any). Example for simple static site:`

12. `cp -r $WORKSPACE/* /var/www/html/` `# assumes permissions OK`

13.

14. `# 3. (Optional) Restart webserver if needed`

15. `sudo systemctl restart nginx` `# or apache2`

16.

17. `echo "Deployment finished"`

- Use `$WORKSPACE` variable (Jenkins supplies it) or absolute paths.
- For production, use rsync over SSH to a deploy host, Docker image builds, or artifact deployments, not plain cp.

18. Configure Webhook on the Git host:

- **GitLab:** Project → Settings → Webhooks → add Jenkins endpoint (depending on plugin; e.g., `http://jenkins-server:8080/generic-webhook-trigger/invoke`) and select *Push events*. Paste any secret token if used. Save and test webhook.
- **GitHub:** Settings → Webhooks → add Jenkins endpoint (or use GitHub App integration).
- **Bitbucket:** Similar webhook settings page.

19. Test flow:

- Make a commit in your editor → push to remote.
- Git host sends webhook → Jenkins receives it and triggers the job → Jenkins executes shell and deploys files to web root.
- Verify by opening `http://your-deploy-host` to confirm the updated site.

Security & Best Practices:

- Use Jenkins Credentials to store SSH keys and reference them in the job instead of embedding secrets.
- Restrict webhook endpoints with tokens and validate payloads.
- Run Jenkins agent with minimal privileges; deployment steps that need root should be done via a controlled deploy user (SSH + sudoers) rather than giving Jenkins full root.

Q.15) Create a new file on a separate branch, make some changes to this file, then merge these changes into main using the Bitbucket web interface.

A:

Goal: Use Bitbucket UI to review and merge branch changes.

Steps:

1. **Create project files in your editor**, commit locally and push to Bitbucket main:
2. `git init`
3. `git add .`
4. `git commit -m "Initial commit"`
5. `git remote add origin <BITBUCKET_REPO_URL>`
6. `git branch -M main`
7. `git push -u origin main`

8. **Create a new branch locally and add a new file (edit in editor):**
 9. `git checkout -b feature/new-file`
 10. # In editor create NEWFILE.md, save
 11. `git add NEWFILE.md`
 12. `git commit -m "Add NEWFILE.md with initial content"`
 13. `git push -u origin feature/new-file`
 14. **Open Bitbucket web UI** → navigate to your repository → you should see the pushed branch.
 15. **Create a Pull Request (PR) / Pull Request UI in Bitbucket:**
 - Click **Create pull request** from feature/new-file → main.
 - Add description, reviewers (optional).
 - Review the changes shown (diff).
 16. **Merge the PR using Bitbucket UI:**
 - Click **Merge** (choose merge strategy if shown).
 - Optionally delete the branch via Bitbucket after a successful merge.
 17. **Update your local main (pull merged changes):**
 18. `git checkout main`
 19. `git pull origin main`
 20. **Show chronological commits (oldest→newest):**
 21. `git log --reverse --pretty=format:'%h %ad | %s [%an]' --date=short`
-

Welcome Mr. Shinde, I'm here for you

Below are **clear, exam-ready Q&A answers** for Q.16 → Q.20. Each answer is concise, follows a consistent editor-first workflow (create/edit files in your IDE — **not** via terminal echo tricks), and gives exact commands you can run for Git/Maven/Docker/Jenkins/Bitbucket tasks.

Q.16) Outline the process of setting up a CI/CD pipeline for a web application using Jenkins, Git, and a local HTTP server. Include the configuration of Jenkins, the webhook setup, and the execution of build and deployment steps.

A:

Goal: Automatically build, test and deploy a web app when code is pushed to the Git repo.

Steps — high level & commands where applicable

1. Prepare the project (in your editor)

- Create project files (index.html, CSS/JS, README.md).
- Initialize Git and push to a remote Git provider (GitHub/GitLab/Bitbucket):
- `git init`
- `git add .`
- `git commit -m "Initial web project"`
- `git branch -M main`
- `git remote add origin <REMOTE_URL>`
- `git push -u origin main`

2. Install & configure Jenkins

- Install Jenkins (package or Docker). Open `http://localhost:8080` and finish setup.
- Install plugins: **Git**, **Pipeline**, **Generic Webhook Trigger** (or provider-specific plugin), **Credentials**.

3. Create a Jenkins job (Pipeline or Freestyle)

- Prefer a **Pipeline** job with a Jenkinsfile stored in your repo (Pipeline-as-code).
- Example Jenkinsfile (declarative):
- `pipeline {`
- `agent any`
- `stages {`
- `stage('Checkout') { steps { checkout scm } }`
- `stage('Build') { steps { echo 'Build (e.g., npm/mvn build)' } }`
- `stage('Test') { steps { echo 'Run tests' } }`
- `stage('Deploy') {`
- `steps {`
- `// Copy files to local HTTP server doc root`
- `sh 'cp -r $WORKSPACE/* /var/www/html/' // adapt path for nginx or`
`apache`

- }
- }
- }
- post { always { archiveArtifacts artifacts: '**/*', allowEmptyArchive: true } }
- }
- Ensure Jenkins user can write to the web root (use a deploy user or configure sudoers/SSH).

4. Configure webhook on Git host

- In repo settings (GitHub/GitLab/Bitbucket), add a webhook URL pointing to Jenkins trigger endpoint (depends on plugin).
 - Example Generic Webhook Trigger endpoint: `http://<jenkins-host>:8080/generic-webhook-trigger/invoke`
- Select events: push (and optionally merge request / PR events). Add secret token if supported.

5. Secure credentials

- Add SSH keys or user tokens to Jenkins **Credentials** and use them in the job instead of plaintext.

6. Test pipeline

- Make an editor change → git add → git commit → git push.
- Webhook triggers Jenkins → job runs stages → deploy stage copies files to `/var/www/html/`.
- Verify site at `http://localhost` (or `http://localhost:8080` depending on server).

7. Best practices

- Run tests before deploy. Use artifact storage (build → archive → deploy) rather than copying workspace. Use separate deploy user and rollback plan.

Q.17) Implement Bitbucket Operations Using Git

Task 1: Create new repository on Bitbucket and clone it locally.

A (Task 1):

1. Create repository on Bitbucket web UI → copy HTTPS/SSH URL.
2. Clone locally:

3. `git clone <BITBUCKET_REPO_URL>`

4. `cd <repo-folder>`

Task 2: Create example.txt, add & commit, create branch feature.

A (Task 2):

- Create example.txt in your editor, save.
- Stage & commit:
- `git add example.txt`
- `git commit -m "Add example.txt"`
- Create feature branch:
- `git checkout -b feature`

Task 3: Push feature to Bitbucket and create a pull request; review & merge.

A (Task 3):

- Push branch:
- `git push -u origin feature`
- On Bitbucket UI → create a Pull Request from feature → main.
- Review changes in UI and **Merge** the PR. Optionally delete the branch after merge.
- Update local main:
- `git checkout main`
- `git pull origin main`

Q.18) Explain how to set up and manage dependencies in a Maven project. Describe the structure of a pom.xml file, adding dependencies, and how Maven handles build automation (compile, test, generate JAR).

A:

1. pom.xml structure — core elements

```
<project xmlns="http://maven.apache.org/POM/4.0.0" ...>
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.example</groupId>
```

```
<artifactId>myapp</artifactId>
```

<version>1.0-SNAPSHOT</version>

<packaging>jar</packaging>

<properties>

<maven.compiler.source>17</maven.compiler.source>

<maven.compiler.target>17</maven.compiler.target>

</properties>

<dependencies>

<!-- project dependencies go here -->

</dependencies>

<build>

<plugins>

<!-- compiler, surefire, shade, etc. -->

</plugins>

</build>

</project>

2. Adding dependencies

- Add a <dependency> block inside <dependencies> with groupId, artifactId, and version:

<dependency>

<groupId>org.apache.commons</groupId>

<artifactId>commons-lang3</artifactId>

<version>3.12.0</version>

</dependency>

- Maven downloads dependencies from remote repositories (Maven Central) and stores them in the local repo (~/.m2/repository).

3. Build automation — standard lifecycles and commands

- mvn compile → compiles source code (src/main/java).

- mvn test → runs unit tests (src/test/java), using Surefire plugin.
- mvn package → compiles + tests + packages into artifact (JAR/WAR) in target/.
- mvn install → installs artifact to local Maven repository for other projects to use.
- Plugins manage additional tasks: maven-compiler-plugin, maven-surefire-plugin, maven-jar-plugin, maven-shade-plugin.

4. Dependency scopes

- compile (default) — required at compile & runtime.
- provided — compile-time only (container provides at runtime).
- runtime — available at runtime but not at compile-time.
- test — only for tests.

5. Best practices

- Use specific versions (avoid snapshots for releases).
- Use dependency management (<dependencyManagement>) for multi-module projects.
- Use plugins for code quality, testing, and packaging (Surefire, JaCoCo, Checkstyle).

Q.19) Install Docker and create a simple "Hello, World!" HTML app, containerize it using an official web server image, build, tag and run so the app is accessible at <http://localhost:8080>.

A:

1. Install Docker

- Follow official docs for your OS (Docker Desktop for Windows/Mac, Docker Engine for Linux). Verify:
- docker --version

2. Create app in your editor

- index.html:

```
<!DOCTYPE html>

<html>

<head><title>Hello</title></head>

<body><h1>Hello, World!</h1></body>

</html>
```

3. Create Dockerfile (use official nginx image)

FROM nginx:stable

COPY index.html /usr/share/nginx/html/index.html

4. Build and tag the image

docker build -t hello-nginx:1.0 .

5. Run container mapping host port 8080 → container port 80

docker run -d --name hello-container -p 8080:80 hello-nginx:1.0

6. Verify

- Open <http://localhost:8080> → you should see "Hello, World!".

7. Cleanup (stop & remove)

docker stop hello-container

docker rm hello-container

docker rmi hello-nginx:1.0

Q.20) Git and GitHub Repository Management

Task 1: Repository setup and initial commit

A (Task 1):

1. Create project.md in your editor with a brief description.
2. Initialize Git, add & commit, and push to GitHub:
3. `git init`
4. `git add project.md`
5. `git commit -m "Initial commit: add project.md"`
6. `git branch -M main`
7. `git remote add origin https://github.com/your-username/your-repo.git`
8. `git push -u origin main`

Task 2: Branching and merging

A (Task 2):

1. Create and switch to feature-branch:
2. `git checkout -b feature-branch`
3. Edit project.md in your editor (add details) → save. Stage & commit:

4. `git add project.md`
5. `git commit -m "Update project.md: add details"`
6. `git push -u origin feature-branch`
7. Merge into main:
 - Option A (locally):
 - `git checkout main`
 - `git merge feature-branch -m "Merge feature-branch into main"`
 - `git push origin main`
 - Option B (GitHub Pull Request): open PR on GitHub from feature-branch → main, merge via UI, then:
 - `git checkout main`
 - `git pull origin main`
8. Confirm merge on GitHub — commits will appear in the repository history.

Sure — nice, clear **Q&A** for **Q.21 → Q.26**, exam/viva-ready, editor-first (create files in VS Code / Notepad — not via terminal text redirects). Each answer shows the goal, step-by-step actions, exact commands you'll run in the terminal, and short viva notes where useful.

Q.21) Containerize a basic application and deploy it using Docker.

A:

Goal: Build a simple app (static HTML or small web app), create a Docker image, run it as a container and expose it on `http://localhost:8080`.

Steps:

1. **Create app in your editor**
 - `index.html` (example):
 - `<!doctype html>`
 - `<html><head><title>Hello</title></head>`
 - `<body><h1>Hello from Docker!</h1></body></html>`
2. **Create Dockerfile** (editor):
3. `FROM nginx:stable`
4. `COPY index.html /usr/share/nginx/html/index.html`

5. **Build image** (terminal):
6. `docker build -t my-simple-site:1.0 .`
7. **Run container mapping port 8080 → 80:**
8. `docker run -d --name mysite -p 8080:80 my-simple-site:1.0`
9. **Verify:** open `http://localhost:8080` in browser.
10. **Stop & remove** (cleanup):
11. `docker stop mysite`
12. `docker rm mysite`
13. `docker rmi my-simple-site:1.0`

Viva note: docker build creates an image from the Dockerfile; docker run -p maps host ports to container ports so the app becomes accessible locally.

Q.22) Git and GitHub Repository Management

Objective: Manage repos, branching, merging on GitHub.

Task 1 — Repository setup & initial commit

A:

1. Create project.md in your editor and write the project description.
2. In terminal (inside project folder):
3. `git init`
4. `git add project.md`
5. `git commit -m "Initial commit: add project.md"`
6. `git branch -M main`
7. `git remote add origin https://github.com/your-username/your-repo.git`
8. `git push -u origin main`

Task 2 — Branching & merging

A:

1. Create a branch:
2. `git checkout -b feature-branch`
3. Edit project.md in your editor (add details), then:
4. `git add project.md`

5. `git commit -m "Update project.md: add detailed description"`
6. `git push -u origin feature-branch`
7. Merge into main (locally):
8. `git checkout main`
9. `git merge feature-branch -m "Merge feature-branch into main"`
10. `git push origin main`

Or open a Pull Request on GitHub and merge via the web UI, then `git pull origin main` locally.

Viva note: Use `git log --oneline --graph --all` to review history; use `git log --reverse` for oldest→newest chronological order.

Q.23) Applying CI Principles to Web Development Using Jenkins and Git

A:

Goal: Automatically run build and tests on every commit (CI), produce artifacts, and optionally fail the pipeline on test failures.

Steps:

1. **Push project to remote Git repository** (GitHub/GitLab/Bitbucket).
2. **Install & configure Jenkins:**
 - Install Jenkins and plugins: **Git, Pipeline, GitHub/GitLab plugin, Credentials.**
 - Create Jenkins credentials for repo access (SSH key or personal access token).
3. **Create Jenkinsfile** (place in repo root) — example declarative CI pipeline:
4. `pipeline {`
5. `agent any`
6. `stages {`
7. `stage('Checkout') { steps { checkout scm } }`
8. `stage('Build') { steps { echo 'Building... (npm/mvn compile)' } }`
9. `stage('Test') { steps { echo 'Run unit tests here' } }`
10. `}`
11. `post {`
12. `failure { echo 'Notify: build failed' }`
13. `success { echo 'Notify: build passed' }`

14. }

15. }

16. **Configure webhook** on your Git host to notify Jenkins on push (see Q.16/Q.14 steps).

17. **Verify:** push a commit → Jenkins triggers job → tests run → CI status shown in Jenkins and (if configured) on the Git host.

Viva tips: CI focuses on build + test automation. Keep tests fast and deterministic; fail the pipeline when tests fail.

Q.24) Containerize a basic application and deploy it using Docker.

A: (Same as Q.21 — condensed repeat)

1. Create app (index.html) and Dockerfile with official web server (nginx).
2. Build:
3. `docker build -t basic-web:1.0 .`
4. Run:
5. `docker run -d --name basic-web -p 8080:80 basic-web:1.0`
6. Visit `http://localhost:8080`. Stop/remove container and image when done.

Viva note: Show `docker ps` to see running containers and `docker logs <name>` for output.

Q.25) Bitbucket Repository and Branch Management

Task 1 — Repository setup & branching

A:

1. Create a repo on Bitbucket web UI and copy the HTTPS URL.
2. Clone locally:
3. `git clone https://bitbucket.org/your-username/your-repo.git`
4. `cd your-repo`
5. Create development branch and new file (create in editor, e.g., dev-notes.md), then:
6. `git checkout -b development`
7. `git add dev-notes.md`
8. `git commit -m "Add dev-notes on development branch"`
9. `git push -u origin development`

Task 2 — Pull request & code review

A:

1. In Bitbucket UI → create a Pull Request from development → main.
2. Assign a reviewer (or self-review), use the diff to comment on changes.
3. After review, **Merge** the PR in Bitbucket UI and optionally delete the branch.
4. Update local main:
5. git checkout main
6. git pull origin main

Viva note: Bitbucket PRs provide a place to comment line-by-line and require approvals before merging (if enabled).

Q.26) Automated Deployment with Jenkins and GitHub

Task 1 — Configure Jenkins with GitHub

A:

1. Install Jenkins and plugins: **Git, GitHub, Pipeline, Credentials, GitHub Branch Source**.
2. Add GitHub credentials (Personal Access Token) in Jenkins → Credentials.
3. Create a Jenkins Pipeline job and configure it to use the GitHub repo (via repo URL + credentials).
4. In GitHub repo → Settings → Webhooks → add Jenkins webhook endpoint (or use GitHub App integration) so pushes notify Jenkins.

Task 2 — Create CI/CD pipeline that clones, builds, archives artifacts & notifies GitHub

A:

1. **Example Jenkinsfile** that clones repo, builds (example: a static site or mvn package), archives artifact and sets build status on GitHub (using the *GitHub* plugin / "Set GitHub commit status" post-build action or the githubNotify/githubStatus pipeline step if plugin available).
2. pipeline {
3. agent any
4. environment {
5. GITHUB_TOKEN = credentials('github-token-id') // store token in Jenkins Credentials
6. }
7. stages {

```

8.   stage('Checkout') { steps { checkout scm } }
9.   stage('Build') { steps {
10.    // Example: mvn package or npm build
11.    sh 'mvn -q -DskipTests package || true' // adapt to your project
12.  }}
13.  stage('Archive') {
14.    steps {
15.      archiveArtifacts artifacts: 'target/*.jar', fingerprint: true
16.    }
17.  }
18. }
19. post {
20.  success {
21.    // Option A: use plugin step (if available)
22.    // githubNotify context: 'CI','state':'SUCCESS' -- requires plugin
23.    echo "Build succeeded"
24.  }
25.  failure {
26.    echo "Build failed"
27.  }
28. }
29. }

```

30. Set GitHub build status:

- Install "**GitHub Integration Plugin**" or "**GitHub Checks**" in Jenkins and configure a post-build action to update the commit/status on GitHub.
- Alternatively, use a simple curl in post to call GitHub's Statuses API (requires token) — plugin-based approaches are preferred.

31. **Test the pipeline:** push a commit → webhook triggers Jenkins build → Jenkins builds & archives → GitHub shows build status on the commit/PR.

Viva tips / security: store tokens/SSH keys in Jenkins Credentials, not in plain text; use least-privilege tokens and protect your webhook endpoints.

Q.26 — Task 3: Adding Deployment Step & GitHub Webhook

Q: How do I extend the Jenkins pipeline to deploy the built application to a local web server (using shell commands) and set up a GitHub webhook to trigger the Jenkins job automatically on push?

A (clean, exam-ready answer):

Goal

1. Extend the Jenkinsfile to include a **Deploy** stage that runs shell commands to copy the built artifact (or site files) to the local HTTP server document root and restart the server if needed.
 2. Configure GitHub to send a **webhook** to Jenkins so every push starts the pipeline automatically.
-

1) Prepare the target web server & Jenkins permissions (pre-steps)

- Decide how Jenkins will deploy:
 - **Local deploy (Jenkins and web server on same machine)** — Jenkins can copy files into `/var/www/html` (Apache) or `/usr/share/nginx/html` (Nginx).
 - **Remote deploy (Jenkins on one host, web server on another)** — use SSH/rsync from Jenkins to the remote server.
- **Security & permission notes:**
 - Prefer a deploy user or let Jenkins use an SSH key to copy files to the web server instead of running Jenkins as root.
 - If using local sudo for file copy or restarting services, configure a safe sudoers rule for Jenkins, e.g.:
 - # on the server (as root), create a sudoers file:
 - `echo 'jenkins ALL=(ALL) NOPASSWD: /bin/cp, /bin/mv, /bin/systemctl restart nginx, /bin/systemctl restart apache2' \`
 - `> /etc/sudoers.d/jenkins_deploy`
 - `chmod 440 /etc/sudoers.d/jenkins_deploy`

(Only grant the minimal commands needed. Prefer SSH/rsync over giving broad privileges.)

2) Example Jenkinsfile (declarative) — Local deployment using shell

Put this Jenkinsfile in your repository root. It checks out code, builds (example mvn package), archives artifacts, then deploys using shell commands.

```
pipeline {
  agent any

  environment {
    WEBROOT = "/var/www/html"      // change to your web server root
    ARTIFACT = "target/myapp-1.0-SNAPSHOT.jar" // example artifact path
  }

  stages {
    stage('Checkout') {
      steps { checkout scm }
    }

    stage('Build') {
      steps {
        // adapt to your build tool: maven/npm/gradle...
        sh 'mvn -B -DskipTests package'
      }
    }

    stage('Test') {
      steps {
        sh 'mvn test'
      }
    }
  }
}
```



```
stage('Archive') {  
    steps {  
        archiveArtifacts artifacts: 'target/*.jar', fingerprint: true  
    }  
}
```

```
stage('Deploy') {  
    steps {  
        script {  
            // Example: deploy built web files or jar-based web app  
            // Option A: Copy static site files (if your build produces site/)  
            // sh "sudo cp -r ${WORKSPACE}/site/* ${WEBROOT}/"  
  
            // Option B: Copy artifact (example for a simple static site)  
            sh """  
                echo 'Starting deployment...'  
                sudo cp -v ${WORKSPACE}/${ARTIFACT} ${WEBROOT}/ || true  
                # if you need to restart the web server:  
                sudo systemctl restart nginx || true  
                echo 'Deployment complete.'  
            """  
        }  
    }  
}
```

```
post {  
    success { echo 'Pipeline succeeded.' }
```

```
failure { echo 'Pipeline failed.' }  
}  
}
```

Notes:

- Replace mvn steps with your actual build commands (e.g., npm run build).
- Replace ARTIFACT and WEBROOT paths to match your project and server.
- sudo is used here for a local server; prefer using SSH to a separate deploy user for production.

3) Alternative: Deploy via SSH (recommended for remote servers)

Why: avoids giving Jenkins sudo on the server. Use SSH credentials stored in Jenkins.

Steps:

1. Generate an SSH key in Jenkins (or a dedicated agent) and add the public key to the ~deploy_user/.ssh/authorized_keys of the web server.
2. Add the SSH private key as a Jenkins credential (type: SSH Username with private key).
3. Use sshagent in Jenkinsfile to run rsync or scp:

```
stage('Deploy via SSH') {  
  steps {  
    sshagent (credentials: ['jenkins-ssh-key-id']) {  
      sh '''  
        echo "Deploying artifacts to remote server..."  
        rsync -avz --delete ${WORKSPACE}/site/  
deployuser@webserver.example.com:/var/www/html/  
        ssh deployuser@webserver.example.com "sudo systemctl restart nginx"  
        '''  
      }  
    }  
  }  
}
```

4) Configure Jenkins job to allow webhook triggers

- If you're using a **Pipeline job** with Jenkinsfile:
 - In job configuration → **Build Triggers**:
 - If using GitHub plugin: enable "**GitHub hook trigger for GITScm polling**".
 - If using Generic Webhook Trigger plugin: configure the plugin and define tokens/filters.
 - Ensure Jenkins is reachable by GitHub (publicly accessible or use tunneling like ngrok for local Jenkins during testing).
 - If Jenkins is behind firewall or local only, you must expose its webhook endpoint or use GitHub Actions/other approaches.
-

5) Set up the GitHub webhook

1. In your GitHub repository → **Settings** → **Webhooks** → **Add webhook**.
2. Fill in:
 - **Payload URL**:
 - If using GitHub plugin (recommended):
http://<JENKINS_HOST>:8080/github-webhook/
 - If using Generic Webhook Trigger plugin:
http://<JENKINS_HOST>:8080/generic-webhook-trigger/invoke?token=MYTOKEN
 - **Content type**: application/json
 - **Secret**: (optional but recommended) a secret string — configure the same token/secret in Jenkins plugin/job.
 - **Which events**: choose **Just the push event** (or add PR events if you want builds for PRs too).
3. Click **Add webhook** and test it using "**Recent Deliveries**" → **Redeliver** to check response.

Important: If GitHub cannot reach your local Jenkins (connection timeout), use one of:

- Expose Jenkins via a public server/IP, or
 - Use ngrok to create a temporary public URL: ngrok http 8080 and use that URL as webhook endpoint, or
 - Use a CI/CD SaaS integration.
-

6) Test the full flow

1. Make a change in your repository (edit a file in your editor) → git add → git commit → git push origin main.
 2. GitHub sends a webhook to Jenkins → Jenkins triggers the pipeline automatically.
 3. Pipeline runs stages: checkout → build → test → archive → deploy.
 4. Verify deployment:
 - Open `http://<web-server-host>/` (or `http://localhost` if local) and confirm the update.
 - Check Jenkins console log for the Deploy stage output.
-

7) Troubleshooting & tips

- **Webhook delivery failed:** inspect GitHub webhook delivery logs for HTTP response codes and Jenkins logs (`/var/log/jenkins/jenkins.log` or job console).
 - **403/401 from Jenkins:** ensure CSRF protection and tokens are configured correctly; GitHub plugin typically handles this. For Generic Webhook plugin, include the token query parameter or header.
 - **Permissions errors when copying files:** adjust sudoers or prefer SSH/rsync as shown.
 - **Use artifacts, not workspace:** archive your build and deploy the archived artifact so builds are reproducible.
 - **Keep secrets safe:** store GitHub tokens and SSH keys in Jenkins Credentials, not in Jenkinsfile.
-

8) Short viva answers (one-line)

- **How does webhook trigger Jenkins?** GitHub sends an HTTP POST to a Jenkins endpoint (e.g., `/github-webhook/`), Jenkins receives it and triggers the configured job.
- **How to deploy safely?** Use SSH + deploy user/rsync; avoid running Jenkins as root; restrict sudo to minimal commands if needed.
- **How to verify deployment?** Check the Jenkins deploy stage console + open the web server URL.

Welcome Mr. Shinde, I'm here for you

Below are **clean, exam/viva-ready Q&A** answers for **Q.27 → Q.30**.

I kept each answer short, precise, editor-first (create/edit files in your IDE — **do not** use terminal echo/redirect tricks), and included the exact commands you'll run in the terminal where needed.

Q.27) Bitbucket Repository Management

Q: How do I create and configure a Bitbucket repository, set access controls, work with a branch, and merge changes?

A:

Task 1 — Create & push README

1. **Create repo on Bitbucket (web UI):**
 - Go to Bitbucket → Create repository → enter **name** (meaningful) and **description** → Create.
2. **Clone locally:**
3. `git clone https://bitbucket.org/your-username/your-repo.git`
4. `cd your-repo`
5. **In your editor:** create README.md and add content (project title + short description).
6. **Push README:**
7. `git add README.md`
8. `git commit -m "Add README with project description"`
9. `git push -u origin main`

Task 2 — Access controls

1. **Invite collaborator (Read access):**
 - Bitbucket repo → Settings → User and group access → Invite user (enter their Bitbucket username or email) → select **Read** → Add.
2. **Upgrade to Write access:**
 - In the same **User and group access** area change the collaborator's role to **Write** (select and update).
3. **Confirm collaborator can push:**
 - Ask collaborator to clone the repo, make a small edit in editor (e.g., edit README), `git add` → `git commit` → `git push origin main` — or push to their branch and open PR.
4. **Document the change:**
 - In your editor create ACCESS_CHANGE.md describing the access update (who, when, old role, new role).

- Push:
- `git add ACCESS_CHANGE.md`
- `git commit -m "Document access control changes: upgraded collaborator from Read → Write"`
- `git push origin main`

Task 3 — Branch workflow and PR

1. **Create branch:**
2. `git checkout -b feature/update-readme`
3. **Edit README.md in editor** (minor update) and push:
4. `git add README.md`
5. `git commit -m "Update README: small content change on feature/update-readme"`
6. `git push -u origin feature/update-readme`
7. **Create Pull Request (Bitbucket UI):**
 - Open repo → Create pull request → source feature/update-readme → target main → add description → create PR.
8. **Review & Merge:** review diff in UI → Merge PR → optionally delete branch.
9. **Update local main:**
10. `git checkout main`
11. `git pull origin main`

Q.28) Applying CI/CD Principles to Web Development Using Jenkins, Git and Local HTTP Server

Q: How do I apply CI/CD (Jenkins + Git + Apache/Nginx) for a web app?

A:

Goal: On every push: checkout → build/test → deploy to local web server.

Steps (concise):

1. **Prepare repo:** create web files in your editor (index.html, assets), push to Git remote.
2. **Install Jenkins & plugins:** install Jenkins; add **Git**, **Pipeline**, and **Generic Webhook Trigger** or provider plugin.
3. **Add Jenkinsfile** to repo (example skeleton):

4. pipeline {
5. agent any
6. stages {
7. stage('Checkout') { steps { checkout scm } }
8. stage('Build') { steps { echo 'Build (npm/mvn if needed)' } }
9. stage('Test') { steps { echo 'Run tests if any' } }
10. stage('Deploy') {
11. steps {
12. sh 'sudo cp -r \$WORKSPACE/* /var/www/html/' // adapt path
13. sh 'sudo systemctl restart nginx || true'
14. }
15. }
16. }
17. }
18. **Give Jenkins deploy rights:** prefer SSH deploy user or minimal sudo rules for Jenkins (example sudoers entry for limited commands).
19. **Configure webhook:** repo settings → Webhooks → add Jenkins endpoint (e.g. /generic-webhook-trigger/invoke?token=... or /github-webhook/), select push events.
20. **Test:** push change → webhook triggers Jenkins → pipeline runs → deployed files appear at http://localhost (or server host).

Viva tips: Use Pipeline-as-code (Jenkinsfile), store credentials in Jenkins Credentials, and deploy via SSH/rsync for remote servers.

Q.29) Bitbucket Repository and Branch Management

Q: How do I create a Bitbucket repo, create a development branch, push changes, and perform PR with review?

A:

Task 1 — Repo & branch

1. **Create repo on Bitbucket** (web UI).
2. **Clone locally:**
3. `git clone https://bitbucket.org/your-username/your-repo.git`

4. `cd your-repo`
5. **Create development branch, add file, commit & push:**
6. `git checkout -b development`
7. # create file in editor: `dev-notes.md`
8. `git add dev-notes.md`
9. `git commit -m "Add dev-notes on development branch"`
10. `git push -u origin development`

Task 2 — PR & review

1. **Open Pull Request** in Bitbucket: source = development, target = main.
2. **Assign reviewer** in PR (or self-review). Use the Bitbucket diff/comment UI to add line-by-line comments.
3. **Address comments** (if any) by editing on development branch locally, commit & push.
4. **Merge PR** after approval → optionally delete branch.
5. **Update local main:**
6. `git checkout main`
7. `git pull origin main`

Q.30) Create a simple project using any programming language and perform some operation on the project using Git (init, stage, commit with descriptive message)

Q: How do I create a small project and perform basic Git operations (make repo, stage file, commit with descriptive message)?

A:

Steps (editor-first):

1. **Create project in your editor** (example Python):
 - `app.py`:
 - `print("Hello, Git and GitHub!")`
 - `README.md` — short description.
2. **Initialize git repo (terminal):**
3. `cd path/to/project-folder`
4. `git init`

5. **Stage files:**

6. `git add app.py README.md`

- or `git add .` to stage all files.

7. **Commit with descriptive message:**

8. `git commit -m "Initial commit: add app.py and README describing project purpose"`

9. **(Optional) Create remote & push:**

- Create remote repository (GitHub/Bitbucket/GitLab).
- Add remote and push:
- `git branch -M main`
- `git remote add origin https://github.com/your-username/your-repo.git`
- `git push -u origin main`

Viva-ready points:

- Staging area holds files before commit.
- Commit message should be concise and descriptive (what and why).
- Use `git status` to check staged/unstaged files and `git log --oneline` to view commit history.

Welcome Mr. Shinde, I'm here for you

Below are complete, **step-by-step, copy-paste-ready** instructions for **Q.31 → Q.35** so you can perform every practical without needing any other guidance.

I assume you will *create and edit files in your editor (VS Code/IntelliJ/Notepad)* — I will **not** use terminal tricks like `echo > file`. Commands shown are real Git/Maven/Docker/Jenkins commands you will run in a terminal where appropriate.

Q.31 Create a simple Java project using Maven — add dependencies, configure POM, and compile

Goal: Create a minimal Maven Java project, add a dependency (Gson), configure `pom.xml`, compile with Maven and run.

1. Create the project

Option A — use IDE (recommended):

- In IntelliJ: New → Maven Project → set `GroupId = com.example`, `ArtifactId = myapp`.

- In VS Code: create the folder myapp, then create pom.xml (see below) and src folders.

Option B — use Maven archetype:

```
mvn archetype:generate -DgroupId=com.example -DartifactId=myapp \
-DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
cd myapp
```

2. Project structure (what to have in editor)

```
myapp/
  pom.xml
  src/
    main/
      java/
        com/example/App.java
    test/
      java/
        com/example/AppTest.java
```

3. App.java (open in editor)

```
package com.example;

import com.google.gson.Gson;

public class App {
    public static void main(String[] args) {
        Message msg = new Message("Hello Maven + Gson!");
        Gson gson = new Gson();
        String json = gson.toJson(msg);
        System.out.println("JSON: " + json);
    }
}
```

```

class Message {
    private String message;

    public Message(String message) { this.message = message; }

    public String getMessage() { return message; }
}

```

4. pom.xml (editor — exact content)

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>myapp</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
    </properties>

    <dependencies>
        <!-- Gson for JSON serialization -->
        <dependency>
            <groupId>com.google.code.gson</groupId>
            <artifactId>gson</artifactId>
            <version>2.11.0</version>
        </dependency>
    </dependencies>

```

```
<!-- JUnit Jupiter for tests -->
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-api</artifactId>
  <version>5.9.2</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-engine</artifactId>
  <version>5.9.2</version>
  <scope>test</scope>
</dependency>
</dependencies>

<build>
  <plugins>
    <!-- Compiler plugin -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.11.0</version>
      <configuration>
        <source>${maven.compiler.source}</source>
        <target>${maven.compiler.target}</target>
      </configuration>
    </plugin>
```

```

<!-- Surefire plugin for running tests -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.0.0-M7</version>
</plugin>

<!-- Optional: maven-jar-plugin to customize JAR -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-jar-plugin</artifactId>
  <version>3.3.0</version>
  <configuration>
    <archive>
      <manifest>
        <mainClass>com.example.App</mainClass>
      </manifest>
    </archive>
  </configuration>
</plugin>
</plugins>
</build>
</project>

```

5. Compile, test, package (run these in terminal inside myapp/)

```

mvn clean compile    # compile source
mvn test             # run tests (if you added any)
mvn package          # build jar in target/myapp-1.0-SNAPSHOT.jar

```

6. Run the JAR

```
java -jar target/myapp-1.0-SNAPSHOT.jar
```

Output should show JSON printed (from Gson).

Q.32 Apply CI/CD Principles to Web Development Using Jenkins, Git and Local HTTP Server

Goal: Build a pipeline that checks out code, builds/tests if needed, and deploys a static site to a local web server (Nginx/Apache) automatically on push.

1. Prerequisites

- Git repository with your web project (index.html, assets).
- Jenkins installed and reachable (e.g., `http://localhost:8080`).
- Local web server installed: **Apache** (`/var/www/html`) or **Nginx** (`/usr/share/nginx/html`).
- Jenkins must have permission to deploy (either via SSH or limited sudo).

2. Add Jenkinsfile to repository (editor)

Example Jenkinsfile (declarative):

```
pipeline {
    agent any

    environment {
        WEBROOT = "/var/www/html"    // change to nginx path if needed
    }

    stages {
        stage('Checkout') {
            steps { checkout scm }
        }

        stage('Build') {
            steps {
                // static site: may be no-op; for frameworks use npm build / hugo etc.
                echo "No build for static HTML"
            }
        }
    }
}
```

```
}
```

```
stage('Test') {  
  steps {  
    echo "No tests for static site (add if you have)"  
  }  
}
```

```
stage('Deploy') {  
  steps {  
    script {  
      // copy content to webroot using sudo (or use SSH/rsync)  
      sh """  
        echo 'Deploying to ${env.WEBROOT} ...'  
        sudo rm -rf ${env.WEBROOT}/* || true  
        sudo cp -r ${WORKSPACE}/* ${env.WEBROOT}/  
        sudo systemctl restart nginx || sudo systemctl restart apache2 || true  
        echo 'Deployment finished'  
      """  
    }  
  }  
}
```

```
post {  
  success { echo 'Pipeline passed' }  
  failure { echo 'Pipeline failed' }  
}  
}
```

Important: Instead of giving Jenkins full sudo, it's better to use SSH to a deploy user or restrict sudo to specific commands (see sudoers approach below).

3. Give Jenkins deploy permissions (recommended safe options)

A. SSH deploy (recommended)

- Create deploy user on the webserver, put Jenkins public key in `~deploy/.ssh/authorized_keys`.
- Use `sshagent` step in Jenkinsfile and `rsync` to copy files.

B. Minimal sudo (if Jenkins on same machine)

- On server as root:

```
echo 'jenkins ALL=(ALL) NOPASSWD: /bin/cp, /bin/rm, /bin/mv, /bin/systemctl restart nginx, /bin/systemctl restart apache2' \
```

```
> /etc/sudoers.d/jenkins_deploy
```

```
chmod 440 /etc/sudoers.d/jenkins_deploy
```

4. Create Jenkins Pipeline job

- New Item → Pipeline → set SCM to your repo or use Jenkinsfile from SCM.
- Add credentials for Git if private (Username/password or SSH key).
- Configure Build Triggers (webhook).

5. Webhook from Git host

- In GitHub/GitLab/Bitbucket repo settings → Webhooks → add Jenkins endpoint:
 - GitHub: `http://<JENKINS_HOST>:8080/github-webhook/`
 - Generic: `http://<JENKINS_HOST>:8080/generic-webhook-trigger/invoke?token=...`
- Choose push events; add secret/token matching Jenkins job config.

6. Test

- Commit & push changes → webhook triggers Jenkins → pipeline runs and deploys.
- Open `http://localhost` (or server IP) and verify site.

Q.33 Bitbucket Repository & Branch Management (complete practical)

Task 1: Setup & Branching

1. **Create repo on Bitbucket** (web UI): name e.g. project-bitbucket, add description.

2. Clone locally:

```
git clone https://bitbucket.org/your-username/project-bitbucket.git
```

```
cd project-bitbucket
```

3. **In editor:** create file dev-notes.md with content.

4. Commit & push on development branch:

```
git checkout -b development
```

```
git add dev-notes.md
```

```
git commit -m "Add dev-notes on development branch"
```

```
git push -u origin development
```

Task 2: Pull Request & Review

1. In Bitbucket web UI → create Pull Request: source development → target main.
2. Assign reviewer (or pick yourself), add comments in PR for review.
3. If changes requested: edit locally on development, commit, push — the PR updates automatically.
4. After approval → Merge PR → delete branch if desired.
5. Update local main:

```
git checkout main
```

```
git pull origin main
```

Q.34 Create a simple project, push to GitHub, perform operations, and display chronological history

Goal: Create local project, init git, commit, push to GitHub, create branch, merge back, show chronological (oldest→newest) history.

1. Create files in editor

- app.py

```
print("Hello from simple GitHub project")
```

- README.md with description.

2. Init, add, commit

```
cd your-project-folder
```

```
git init
```

```
git add app.py README.md
```

```
git commit -m "Initial commit: add app.py and README"
```

3. Create GitHub repo (web), add remote & push

```
git branch -M main
```

```
git remote add origin https://github.com/your-username/your-repo.git
```

```
git push -u origin main
```

4. Create a branch, change, push, merge

```
git checkout -b feature/version-print
```

```
# edit app.py in editor: add print("Version 1.0")
```

```
git add app.py
```

```
git commit -m "Add version print in app.py"
```

```
git push -u origin feature/version-print
```

```
# Option A: Merge locally
```

```
git checkout main
```

```
git merge feature/version-print -m "Merge feature/version-print into main"
```

```
git push origin main
```

```
# Option B: Open GitHub Pull Request and merge there, then:
```

```
git checkout main
```

```
git pull origin main
```

5. Show chronological history (oldest → newest)

```
git log --reverse --pretty=format:'%h %ad | %s [%an]' --date=short
```

This prints commits from the earliest to the latest — perfect for exam demonstration.

Q.35 Applying CI Principles to Web Development Using Jenkins & Git

Goal: Setup Jenkins to run continuous integration (build + tests) on every commit and report results.

1. Push project to Git (GitHub/GitLab/Bitbucket)

- Ensure repo contains either build tool config (Maven pom.xml, npm package.json) or a simple Jenkinsfile that runs tests.

2. Jenkins setup (install plugins)

- Install Jenkins, then install these plugins: **Git, Pipeline, GitHub/GitLab plugin, Credentials, Blue Ocean (optional).**

3. Create a Jenkinsfile (CI-only example)

Place in repo root:

```
pipeline {
  agent any

  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Build') {
      steps {
        // example for maven
        sh 'mvn -B -DskipTests=false test package'
      }
    }
    stage('Static Analysis') {
      steps {
        echo 'Run linters or static checks if present'
      }
    }
  }

  post {
    success {
      echo 'CI passed: tests green'
    }
    failure {
      echo 'CI failed: tests failing'
    }
  }
}
```

```
}  
  
}  
  
}
```

4. Configure job & webhook

- Create a Jenkins Pipeline job (use repo + Jenkinsfile).
- Add Git credentials if private.
- Add webhook on Git host to Jenkins (push events) so pushes trigger the CI:
 - GitHub: `http://<JENKINS_HOST>:8080/github-webhook/`
 - GitLab: use GitLab plugin endpoint or generic webhook.

5. Verify

- Push a commit that causes tests to pass/fail and watch Jenkins build.
- Look at console logs for test output. Fix failing tests and push to demonstrate CI loop.

Final tips & checklist (one page)

- Always **create files in your editor**, save, then use `git add / git commit`.
- Use descriptive commits: "Fix: adjust footer text — explain why".
- For Maven: `mvn clean compile, mvn test, mvn package`.
- For Docker static sites: Dockerfile with `FROM nginx:stable` and `COPY index.html ...`, then `docker build -t name:tag .` and `docker run -d -p 8080:80 name:tag`.
- Git chronological history:
`git log --reverse --pretty=format:'%h %ad | %s [%an]' --date=short`
- Jenkins best practice: store secrets in Jenkins Credentials, prefer SSH deployments, and keep Jenkinsfile versioned in the repo.
- When deploying locally, prefer SSH/rsync to avoid giving Jenkins broad sudo. If you must use sudo, restrict allowed commands in `/etc/sudoers.d/jenkins_deploy`.

Welcome Mr. Shinde, I'm here for you

Below are copy-paste-ready, **complete step-by-step** Q&A answers for **Q.36 → Q.40** so you can perform each practical without needing anything else.

I assume you create and edit files in your code editor (VS Code / IntelliJ / Notepad) — **do not** use terminal echo/redirect tricks. All terminal commands shown are the ones you will run.

Q.36) Git & GitHub — Repository setup, branching, merging (full practical)

Q: How do I set up a local repo, push to GitHub, create feature-branch, merge to main, and push the merged main?

A (complete steps):

Goal: Create project.md → commit → push → branch feature-branch → change → merge → push.

1. Create files in editor

- Create folder project-36 and inside it create project.md with a short description (title + 2–3 lines).

2. Initialize local Git repo (terminal)

3. `cd path/to/project-36`

4. `git init`

5. `git add project.md`

6. `git commit -m "Initial commit: add project.md with project description"`

7. Create remote repo on GitHub

- On GitHub: New → Repository → name e.g. project-36 → Create.
- Copy the HTTPS URL shown.

8. Add remote and push

9. `git branch -M main`

10. `git remote add origin https://github.com/your-username/project-36.git`

11. `git push -u origin main`

12. Create feature branch and make changes (editor)

- In editor open project.md and add a paragraph or bullet points.

13. `git checkout -b feature-branch`

14. `git add project.md`

15. `git commit -m "Feature: add detailed description to project.md"`

16. `git push -u origin feature-branch`

17. Merge feature-branch into main (local method)

18. `git checkout main`

19. `git merge feature-branch -m "Merge feature-branch: update project.md"`

20. git push origin main

Alternatively: create a GitHub Pull Request from feature-branch → main, review and Merge there; then git pull origin main locally.

21. **View chronological (oldest→newest) commit history**

22. git log --reverse --pretty=format:'%h %ad | %s [%an]' --date=short

Done. Your main on GitHub now includes the merged changes.

Q.37) Git & GitHub — (Same practical as Q.36)

Q: Same as Q.36 — how to do full repo → branch → merge → push workflow?

A: Q.37 is identical to Q.36. Repeat the exact steps above (create project.md in an editor, git init → commit → push, create feature-branch → commit → merge → push). Use the git log --reverse ... command to show chronological history.

Q.38) Create a simple project, push to GitHub, create branch, merge into main, show chronological commits

Q: Give me a full, no-miss steps flow for this practical.

A (complete steps):

1. **Create project files in editor**

- Folder simple-web (or language of your choice). Example index.html with a heading and README.md.

2. **Initialize & first commit**

3. cd path/to/simple-web

4. git init

5. git add index.html README.md

6. git commit -m "Initial commit: add index.html and README"

7. **Push to GitHub**

- Create GitHub repo simple-web, then:

8. git branch -M main

9. git remote add origin https://github.com/your-username/simple-web.git

10. git push -u origin main

11. **Create new branch, edit, commit & push**

12. `git checkout -b feature/add-footer`

13. # edit index.html in editor: add footer content, save

14. `git add index.html`

15. `git commit -m "Add footer to index.html"`

16. `git push -u origin feature/add-footer`

17. Merge branch into main

- Option A (local merge):
- `git checkout main`
- `git merge feature/add-footer -m "Merge feature/add-footer into main"`
- `git push origin main`
- Option B (GitHub PR): open Pull Request on GitHub, merge there, then locally:
- `git checkout main`
- `git pull origin main`

18. Show chronological commits (oldest→newest)

19. `git log --reverse --pretty=format:'%h %ad | %s [%an]' --date=short`

Notes: Use `git status` anytime to check staged/unstaged files.

Q.39) Applying CI Principles to Web Development Using Jenkins & Git

Q: How to implement CI (build + tests + feedback) for a web project using Jenkins and Git?
Provide every step.

A (complete, no-miss instructions):

Goal: On every push to the repository Jenkins runs build & tests and reports results.

1. Prepare project repo

- Ensure repository contains a build/test step (if static site, add a simple test script; if Maven/Node project, ensure pom.xml or package.json exists).
- Example (Node): add package.json with "test": "echo 'No tests' && exit 0" for demo.

2. Install Jenkins

- Install Jenkins (official docs) or run Docker:
- `docker run -d -p 8080:8080 -p 50000:50000 --name jenkins jenkins/jenkins:lts`

- Open <http://localhost:8080>, finish setup, install recommended plugins.

3. Install required Jenkins plugins

- At minimum: **Git**, **Pipeline**, **Credentials**, and provider plugin (GitHub plugin) if using GitHub.

4. Add repository credentials in Jenkins

- Jenkins → Credentials → add GitHub token or SSH key so Jenkins can access private repos.

5. Create a Jenkinsfile in project root (example for Maven or Node):

Maven example

```
pipeline {
  agent any
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Build') { steps { sh 'mvn -B -DskipTests=false package' } }
    stage('Test') { steps { sh 'mvn test' } }
  }
  post {
    success { echo 'CI: build & tests passed' }
    failure { echo 'CI: build or tests failed' }
  }
}
```

Node example

```
pipeline {
  agent any
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Install') { steps { sh 'npm install' } }
    stage('Test') { steps { sh 'npm test' } }
  }
}
```


}

6. Create Pipeline job

- Jenkins → New Item → Pipeline → point it to your repo (SCM) and Jenkinsfile or use Multibranch Pipeline for PRs/branches.

7. Configure webhook on Git host (GitHub)

- GitHub repo → Settings → Webhooks → Add webhook:
 - Payload URL: `http://<JENKINS_HOST>:8080/github-webhook/`
 - Content type: `application/json`
 - Events: select **Push** (and PR events if desired)
- Save.

8. Test CI

- Edit a file in editor → `git add` → `git commit` → `git push`.
- Jenkins receives webhook → triggers pipeline → console output shows build/test logs.
- If tests pass, Jenkins marks build as SUCCESS; if fail, job fails.

9. View results and feedback

- Jenkins job page shows console log and status.
- Optionally enable GitHub commit status notifications by installing GitHub plugin and configuring credentials—Jenkins will update commit/PR with build status.

10. Good practices

- Keep tests fast and isolated.
- Fail the build on test failures so developers get quick feedback.
- Use Jenkins Credentials for tokens and keys.
- Use Multibranch Pipeline to automatically create jobs for branches and PRs.

Q.40) Create a project, push to GitHub, create a branch, merge into main, display chronological commits, and pull changes locally

Q: Give me the full workflow including pulling remote merges onto local machine.

A (complete steps):

1. Create project files in editor

- Folder project-40, files project.md and app.py (or any language).

2. **Initialize & push initial commit**

3. `cd path/to/project-40`
4. `git init`
5. `git add project.md app.py`
6. `git commit -m "Initial commit: add project.md and app.py"`
7. # On GitHub create repo project-40 and copy its URL
8. `git branch -M main`
9. `git remote add origin https://github.com/your-username/project-40.git`
10. `git push -u origin main`

11. **Create feature branch, edit, commit & push**

12. `git checkout -b feature/add-contact`
13. # edit files in editor, save
14. `git add .`
15. `git commit -m "Feature: add contact section"`
16. `git push -u origin feature/add-contact`

17. **Merge branch into main (on GitHub via PR recommended)**

- Create a Pull Request on GitHub: source feature/add-contact → target main → Merge.
- Or merge locally:
- `git checkout main`
- `git merge feature/add-contact -m "Merge feature/add-contact into main"`
- `git push origin main`

18. **Pull merged changes to your local machine** (if merge occurred on remote or another machine)

19. `git checkout main`
20. `git pull origin main`

- This updates your local main with the remote merged commit(s).

21. **Show chronological commit history (oldest→newest)**

22. `git log --reverse --pretty=format:'%h %ad | %s [%an]' --date=short`

23. Confirm everything is synced

- `git status` should show clean working tree.
 - `git remote -v` shows remote URL.
 - `git branch -a` shows local and remote branches.
-

Quick reference cheat-commands (copy these)

- Initialize and first commit:
- `git init`
- `git add .`
- `git commit -m "Initial commit: add project files"`
- Set remote and push:
- `git branch -M main`
- `git remote add origin <REMOTE_URL>`
- `git push -u origin main`
- Create branch, commit, push:
- `git checkout -b feature/your-feature`
- # edit files in editor
- `git add .`
- `git commit -m "Feature: describe change"`
- `git push -u origin feature/your-feature`
- Merge local branch into main and push:
- `git checkout main`
- `git merge feature/your-feature -m "Merge feature/your-feature"`
- `git push origin main`
- Pull remote main (sync local):
- `git checkout main`
- `git pull origin main`
- Chronological commits (oldest → newest):

- `git log --reverse --pretty=format:'%h %ad | %s [%an]' --date=short`

Welcome Mr. Shinde, I'm here for you

Below are **complete, step-by-step, copy-paste-ready** answers for **Q.41 → Q.45**. Create/edit files in your editor (VS Code/IntelliJ/Notepad) — **do not** use terminal redirection tricks. All terminal commands shown are real and ready to run. Follow steps in order and you will be able to perform each practical without any other guidance.

Q.41) Create a new file on a separate branch, make some changes to this file, and then merge these changes into the main branch using the Bitbucket interface.

Goal: Use Bitbucket UI to review and merge changes made on a branch.

Steps (complete):

1. Create repo on Bitbucket (web UI)

- Go to Bitbucket → Create repository → give name & description → Create.

2. Clone repository locally

```
git clone https://bitbucket.org/your-username/your-repo.git
```

```
cd your-repo
```

3. Create a new branch locally

```
git checkout -b feature/add-info
```

4. Create a new file in your editor

- Create INFO.md and add some content (title + 3 lines). Save.

5. Stage & commit locally

```
git add INFO.md
```

```
git commit -m "Add INFO.md with project details on feature/add-info"
```

6. Push the branch to Bitbucket

```
git push -u origin feature/add-info
```

7. Open Bitbucket → Create a Pull Request

- Repo → Pull requests → Create pull request → Source: feature/add-info → Target: main → Add title & description → Create.

8. Review in Bitbucket UI

- Use **Diff** view to inspect the new file. Add inline comments if needed. Assign a reviewer (or self-review).

9. Merge via Bitbucket UI

- After review, click **Merge** (choose merge strategy if prompted). Optionally delete branch via checkbox.

10. Update local main

git checkout main

git pull origin main

11. Verify

- git log --oneline --graph --decorate or git log --reverse --pretty=format:'%h %ad | %s [%an]' --date=short to show chronology.

Q.42) Applying CI Principles to Web Development Using Jenkins & Git

Goal: Automate build + tests + fast feedback on every push (CI only, no deploy).

Complete steps:

1. Prepare project

- Ensure repo has a build/test command:
 - Maven: mvn test
 - Node: npm test (define script in package.json)
 - If static site, create a tiny test script scripts/test.sh that returns success/failure.

2. Install Jenkins

- Install Jenkins (package or Docker). Example Docker:

docker run -d -p 8080:8080 -p 50000:50000 --name jenkins jenkins/jenkins:lts

- Open <http://localhost:8080> and finish setup. Install recommended plugins.

3. Install required plugins

- Git, Pipeline, Credentials, (GitHub/GitLab/Bitbucket plugin depending on host), Blue Ocean (optional).

4. Add credentials

- Jenkins → Credentials → add SSH key or Personal Access Token (for private repos).

5. Add Jenkinsfile to repo (CI pipeline)

- Put this file in repo root:

```
pipeline {
  agent any

  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Build') {
      steps {
        // adapt to your tool

        sh 'mvn -B -DskipTests=false package' // Maven example
        // or sh 'npm ci && npm test' for Node
      }
    }
    stage('Test') {
      steps {
        sh 'mvn test' // or 'npm test'
      }
    }
  }

  post {
    success { echo 'CI SUCCESS' }
    failure { echo 'CI FAILED' }
  }
}
```

6. Create Jenkins Pipeline job

- New Item → Pipeline → Configure SCM to point to your repo (or use Multibranch Pipeline).

7. Configure webhook on Git host

- GitHub: http://<JENKINS_HOST>:8080/github-webhook/

- GitLab/Bitbucket: use the provider's Jenkins endpoint or Generic Webhook Trigger plugin endpoint. Set event = push (and PR merge if desired).

8. Test CI

- Edit a file in editor → git add → git commit → git push.
- Jenkins receives webhook, runs pipeline, and shows PASS/FAIL in console and job view.

9. Optional: Report status back to Git host

- Install GitHub plugin and configure credentials so Jenkins updates commit/PR statuses.

Viva one-liners: CI = run build + tests on each commit; use Jenkinsfile to define pipeline as code; use credentials, webhooks for automatic triggering.

Q.43) Create a simple Java project using Maven — add dependencies, configure POM, and compile using Maven

Goal: Minimal Maven Java project with Gson dependency, compile, test, and package a JAR.

File tree (editor):

```
myapp/  
  pom.xml  
  src/  
    main/java/com/example/App.java  
    test/java/com/example/AppTest.java
```

App.java

```
package com.example;  
  
import com.google.gson.Gson;  
  
public class App {  
    public static void main(String[] args) {  
        Message m = new Message("Hello Maven + Gson!");  
        Gson g = new Gson();
```

```
        System.out.println(g.toJson(m));
    }
}
```

```
class Message {
    private String message;

    public Message(String message) { this.message = message; }

    public String getMessage() { return message; }
}
```

pom.xml (copy into editor)

```
<project xmlns="http://maven.apache.org/POM/4.0.0" ...>
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>myapp</artifactId>
    <version>1.0-SNAPSHOT</version>
    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
    </properties>
    <dependencies>
        <dependency>
            <groupId>com.google.code.gson</groupId>
            <artifactId>gson</artifactId>
            <version>2.11.0</version>
        </dependency>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>5.9.2</version>
```



```
<scope>test</scope>
</dependency>
</dependencies>
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.11.0</version>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.0.0-M7</version>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-jar-plugin</artifactId>
      <version>3.3.0</version>
      <configuration>
        <archive>
          <manifest>
            <mainClass>com.example.App</mainClass>
          </manifest>
        </archive>
      </configuration>
    </plugin>
  </plugins>
</build>
```

</project>

Commands (run in myapp/):

mvn clean compile # compiles sources

mvn test # runs unit tests

mvn package # creates target/myapp-1.0-SNAPSHOT.jar

java -jar target/myapp-1.0-SNAPSHOT.jar

Notes: If Java version differs change compiler source/target in pom.xml. Maven downloads dependencies to ~/.m2/repository.

Q.44) Install Docker and create a "Hello, World!" HTML app — containerize with official web server (nginx), build, tag, and run on http://localhost:8080

Goal: Containerize static HTML and expose on port 8080.

1. Install Docker

- Follow Docker Desktop for Windows/Mac or Docker Engine for Linux. Verify:

docker --version

2. In your editor create index.html:

```
<!doctype html>
```

```
<html>
```

```
  <head><title>Hello</title></head>
```

```
  <body><h1>Hello, World!</h1></body>
```

```
</html>
```

3. Create Dockerfile in same folder:

```
FROM nginx:stable
```

```
COPY index.html /usr/share/nginx/html/index.html
```

4. Build the Docker image

```
docker build -t hello-nginx:1.0 .
```

5. Run container mapping port 8080

```
docker run -d --name hello-container -p 8080:80 hello-nginx:1.0
```

6. Verify

- Open browser: `http://localhost:8080` → you should see **Hello, World!**

7. Cleanup

`docker stop hello-container`

`docker rm hello-container`

`docker rmi hello-nginx:1.0`

Q.45) Applying CI Principles to Web Development Using Jenkins & Git (practical, end-to-end CI)

Goal: Full CI: on every push Jenkins runs build & tests and reports results (no deploy).

Complete steps you can follow now:

1. Prepare project with a test command

- For Maven: have `pom.xml` and tests under `src/test/java`
- For Node: `package.json` with "test" script
- For static site: include a small script `scripts/test.sh` that exits 0 or non-zero.

2. Add Jenkinsfile to repo (CI pipeline)

```
pipeline {
  agent any

  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Dependency install') {
      steps {
        // For Maven: nothing needed; for Node: npm ci
        sh 'mvn -B -DskipTests=false package' // change per project
      }
    }
    stage('Test') {
      steps {
        sh 'mvn test' // or 'npm test' or './scripts/test.sh'
      }
    }
  }
}
```

```

}
}
post {
    success { echo 'CI: SUCCESS' }
    failure { echo 'CI: FAILURE' }
}
}

```

3. Create Jenkins job

- New Item → Pipeline → configure SCM / Multibranch Pipeline.

4. Add credentials

- Jenkins → Credentials → add SSH key or token for private repos.

5. Set up webhook

- Git host (GitHub/GitLab/Bitbucket): add webhook pointing to Jenkins trigger endpoint (see Q.42).

6. Run & verify

- Push a commit that causes tests to run. Jenkins will pick it up, run pipeline, tests run in Test stage — console shows results.

7. Optional: Fail-fast

- Configure pipeline to failFast for parallel stages or stop pipeline on first failing test, so feedback is fast.

If you'd like, I can now:

- produce **ready files** (downloadable) for any of these questions (e.g., Jenkinsfile, Dockerfile, pom.xml, App.java, index.html) in a zip, or
- create a **single printable PDF** containing Q.41–Q.45 with all file contents and commands, or
- give a **short viva cheat sheet** summarizing the commands you should memorize for Git / Maven / Docker / Jenkins.

Welcome Mr. Shinde, I'm here for you

Below are **complete, step-by-step, copy-paste-ready** Q&A answers for **Q.46 → Q.50**. Each question includes the goal, exact commands you run in a terminal, and safe editor-first

instructions (create/edit files in your IDE — **do not** use terminal echo/redirect tricks). Follow the steps in order and you'll be able to perform every practical without any other guidance.

Q.46) Create a simple project and push it to GitHub, perform some operations, and display a chronological history of commits

A — Goal: Create a local project, push to GitHub, make a change on a branch, merge, and show commits oldest→newest.

Steps

1. Create files in your editor

- Folder project-46 with app.py:
- `print("Hello from Project 46")`

and README.md with a short description.

2. Initialize repo & first commit

```
cd path/to/project-46
```

```
git init
```

```
git add app.py README.md
```

```
git commit -m "Initial commit: add app.py and README"
```

3. Create GitHub repo (web) and add remote

- Create repo on GitHub named project-46, copy HTTPS URL.

```
git branch -M main
```

```
git remote add origin https://github.com/your-username/project-46.git
```

```
git push -u origin main
```

4. Create a feature branch, edit in editor, commit & push

- In editor: open app.py and add `print("Version 1.0")`, save.

```
git checkout -b feature/add-version
```

```
git add app.py
```

```
git commit -m "Feature: add version print to app.py"
```

```
git push -u origin feature/add-version
```

5. Merge feature branch into main (locally) and push

```
git checkout main
```

```
git merge feature/add-version -m "Merge feature/add-version into main"
```

```
git push origin main
```

(Or merge via GitHub Pull Request and then git pull locally.)

6. Display chronological history (oldest → newest)

```
git log --reverse --pretty=format:'%h %ad | %s [%an]' --date=short
```

Notes: Use git status to check state and git log --oneline --graph --all for a compact visual.

Q.47) Create CI using Webhook and deploy a project using Jenkins Execute shell

A — Goal: Configure Jenkins to be triggered by a webhook and deploy the repo using an Execute shell build step.

Steps

1. Prepare project and repo

- Project with buildable content (e.g., static site: index.html) pushed to GitHub/GitLab/Bitbucket.

2. Install Jenkins & plugins

- Install Jenkins; install plugins: **Git**, **Pipeline** (optional), **Generic Webhook Trigger** (or provider-specific plugin), **Credentials**.

3. Create a Jenkins Freestyle job (simple)

- Jenkins → New Item → Freestyle project → name it → OK.
- **Source Code Management:** select Git and enter repo URL + credentials.
- **Build Triggers:** enable **Trigger builds remotely (e.g., from scripts)** or use **Generic Webhook Trigger** plugin and configure a token.
- **Build → Execute shell:** add shell commands to deploy. Example for static site copying to local Apache/Nginx:
 - `#!/bin/bash`
 - `set -e`
 - `echo "Deploy started at $(date)"`
 - `# workspace contains the checked out repo`
 - `sudo rm -rf /var/www/html/* || true`
 - `sudo cp -r "${WORKSPACE}/*" /var/www/html/`

- `sudo systemctl restart nginx || sudo systemctl restart apache2 || true`
- `echo "Deploy finished at $(date)"`
- Save job.

4. Set up Webhook on Git host

- In repo settings → Webhooks → Add webhook:
 - Payload URL: Jenkins endpoint, e.g.
`http://<JENKINS_HOST>:8080/generic-webhook-trigger/invoke?token=MYTOKEN` (match token configured in job).
 - Content type: `application/json`. Select **Push** events.
- Test webhook with "Test delivery" or push a commit.

5. Permissions / security

- Prefer SSH/rsync to remote server or create a deploy user. If local, restrict sudo to only required commands for Jenkins (see earlier sudoers examples).

6. Test

- Make an editor change → git commit → git push. Webhook triggers Jenkins → Jenkins runs job and Execute shell deploys files. Verify at `http://<server>`.

Notes: For production, prefer pipeline jobs, artifact-based deploys and SSH-based deployments rather than giving Jenkins broad sudo.

Q.48) Applying CI/CD Principles to Web Development Using Jenkins, Git and Local HTTP Server (Apache/Nginx)

A — Goal: Automate build → test → deploy to a local web server on every push (full CI/CD).

Steps

1. Project

- Create web project in editor (index.html, assets). Push to Git host.

2. Install web server

- Install Nginx (`/usr/share/nginx/html`) or Apache (`/var/www/html`) and ensure it serves `http://localhost`.

3. Install Jenkins & plugins

- Jenkins with **Git**, **Pipeline**, **Credentials**, **Generic Webhook Trigger**.

4. Create Jenkinsfile in repository (example declarative pipeline):

```

pipeline {
    agent any

    environment {
        WEBROOT = "/var/www/html"      // change for nginx if needed
    }

    stages {
        stage('Checkout') { steps { checkout scm } }

        stage('Build') {
            steps {
                echo "Build step (if static site, no-op; for frameworks use npm/mvn build)"
            }
        }

        stage('Test') {
            steps {
                echo "Test step (add tests if available)"
            }
        }

        stage('Deploy') {
            steps {
                sh """
                    echo 'Deploying to ${WEBROOT}'
                    sudo rm -rf ${WEBROOT}/* || true
                    sudo cp -r ${WORKSPACE}/* ${WEBROOT}/
                    sudo systemctl restart nginx || sudo systemctl restart apache2 || true
                    echo 'Deployment complete'
                """
            }
        }
    }
}

```



```
post { success { echo 'Pipeline success' } failure { echo 'Pipeline failed' } }
```

5. **Add Jenkins Credentials / SSH keys** if using remote deploy.

6. **Create Jenkins Pipeline job or Multibranch Pipeline**

- Configure SCM to the repo (Jenkinsfile) and add credentials.

7. **Configure webhook on Git host** to trigger the Jenkins job as described in Q.47.

8. **Test end-to-end**

- Make a change → push → webhook triggers Jenkins → pipeline builds/tests → Deploy stage copies artifacts to webroot → Visit <http://localhost> to verify.

Best practices

- Use artifact storage (archiveArtifacts / Nexus) and deploy artifacts, not workspace files.
 - Use SSH & deploy user for remote servers; limit sudo when local.
 - Use branch protections and PR checks for safe merges.
-

Q.49) Create a simple project, push to GitLab, create a new branch, merge into main, and display a chronological history of commits

A — Goal: Full GitLab workflow: create repo, push initial commit, branch, merge, and show oldest→newest commits.

Steps

1. **Create project files in editor**

- Folder project-49 with index.html and README.md.

2. **Initialize local repo & first commit**

```
cd path/to/project-49
```

```
git init
```

```
git add index.html README.md
```

```
git commit -m "Initial commit: add index.html and README"
```

3. **Create GitLab repo (web UI)** and copy HTTPS URL.

4. **Add remote and push**

```
git branch -M main
```

```
git remote add origin https://gitlab.com/your-username/project-49.git
```

```
git push -u origin main
```

5. Create a new branch, edit, commit & push

- Edit index.html in editor (add footer), then:

```
git checkout -b feature/add-footer
```

```
git add index.html
```

```
git commit -m "Add footer to index.html on feature/add-footer"
```

```
git push -u origin feature/add-footer
```

6. Merge via GitLab Merge Request (recommended)

- On GitLab: Create Merge Request feature/add-footer → main, review and **Merge**.
- Or merge locally:

```
git checkout main
```

```
git merge feature/add-footer -m "Merge feature/add-footer"
```

```
git push origin main
```

7. Display chronological commits (oldest → newest)

```
git log --reverse --pretty=format:'%h %ad | %s [%an]' --date=short
```

8. Sync local after MR merge

```
git checkout main
```

```
git pull origin main
```

Notes: GitLab CI can be added later with .gitlab-ci.yml.

Q.50) Create CI using Webhook and deploy a project using Jenkins Execute shell (repeat/complete)

A — Goal: Set Jenkins to be triggered by a webhook and deploy the repository using an Execute shell deploy step.

Steps (same as Q.47 but condensed into a ready checklist):

1. **Project** — push code to a Git host (GitHub/GitLab/Bitbucket).
2. **Jenkins** — install and ensure reachable from your Git host (or use ngrok for local testing).
3. **Create Jenkins job (Freestyle)**
 - SCM: Git repo URL + credentials.

- Build Triggers: enable **Generic Webhook Trigger** (configure token) or **GitHub hook trigger**.
- Build → **Execute shell**: add safe deploy script (example below).

4. **Example safe Execute shell deploy script** (tweak paths & permissions):

```
#!/bin/bash

set -e

echo "Deploy start: $(date)"

# optionally keep a backup

sudo mv /var/www/html /var/www/html.bak.$(date +%s) || true

sudo mkdir -p /var/www/html

sudo cp -r "${WORKSPACE}/*" /var/www/html/

sudo chown -R www-data:www-data /var/www/html || true

sudo systemctl restart nginx || sudo systemctl restart apache2 || true

echo "Deploy complete: $(date)"
```

5. **Webhook on Git host**

- Add webhook pointing to Jenkins job endpoint: e.g.
http://<JENKINS_HOST>:8080/generic-webhook-trigger/invoke?token=MYTOKEN
or use provider-specific webhook URL.

6. **Test flow**

- Push change → webhook fires → Jenkins job runs → Execute shell performs deploy → verify at <http://<server>>.

7. **Post-deploy checks**

- Look at console output in Jenkins, check web server logs
(/var/log/nginx/error.log), verify file ownership and permissions.

Security reminders

- Use Jenkins Credentials for secrets, avoid plaintext tokens.
- Prefer SSH/rsync with a deploy user instead of opening sudo broadly.
- If using sudo, restrict allowed commands in /etc/sudoers.d/ for the jenkins user.